

11.08.2026

Exuplement 31

Aim:

To simulate FIFO page replacement and find the page faults and page hits.

Code:

```
#include <stdio.h>
```

int main() {
int p[20], f[20], n, m, i, j, pos = 0, found = 0;
hit;

scanf("%d", &n);
for (i = 0; i < n; i++)
scanf("%d", &p[i]);

scanf("%d", &m);
for (i = 0; i < m; i++)
f[i] = -1;

for (i = 0; i < m; i++)
f[i] = -1;
for (i = 0; i < m; i++) {
hit = 0;

for (j = 0; j < n; j++)
if (f[j] == p[i])

hit = 1;
if (!hit) {

f[pos] = p[i];

pos = (pos + 1) % m;

faults & t;

3

3 printf ("Page faults = %d", fault);
return 0;

}

Output:

Enter number of pages: 6

Enter page reference string: 1 2 3 1 4 5

Enter number of frames: 3

Page fault = 5

Result:

~~Page 8~~, the FIFO page replacement technique was successfully simulated and the page faults were calculated.

14.08.2026 Experiment - 32

Aim:

To simulate Least Recently Used (LRU) replacement and find page faults.

Code:

```
1) include <stdio.h>  
2) main () {  
    int p[10], f[10], n, m, i, j;  
    int fault = 0, hit;
```

```
    scanf("%d", &n);  
    scanf("%d", &m);
```

```
    for (i = 0; i < n; i++) scanf("%d",  
    &p[i]);
```

```
    for (i = 0; i < m; i++)
```

```
        scanf("%d", &f[i]);
```

```
    for (i = 0; i < m; i++) f[i] = -1;
```

```
    for (i = 0; i < n; i++) {
```

```
        hit = 0;
```

```
        for (j = 0; j < m; j++)
```

```
            if (f[j] == p[i]) {
```

```
                hit = 1; f[j] = i; break;
```

```
    }
```

```
    if (!hit) {
```

```
        fault++;
```

```
        pos = 0;
```

```
        for (j = 1; j < m; j++)
```

```
            if (f[j] == -1 || f[j] < f[pos])  
                pos = j;
```

$f(\text{pos}) = p(\text{pos})$
 $f(\text{pos}) = i$

3

3
printf("Page Faults = %d", fault);
printf("Page Faults = %d, a-faults",
return 0;

3

Output:

Page Faults = 5

Page Hits = 1

Result:

~~Thus~~, the LRU page replacement scheme
was successfully simulated.

14-08-2026 Experiment - 32

Aim:

To simulate optimal page replacement
and find page faults.

Code:

#include <stdio.h>

int main() {

no p[ro]p[er] p[ro]p[er] n. n. d. f. h. e. v. a. s. f. a. c. t. = v. b. i. t. i.

```

scanf("%d", &n);
for(i=0; i<n; i++) scanf("%d", &a[i]);
scanf("%d", &m);
for(i=0; i<m; i++) if(i%2==1);
for(i=0; i<n; i++) {

```

$bit = 0$
 $for (j = 0; j < m; j++)$

if $[f \in F = p[i]] \text{ list} = 1$

$$f(! \text{ list}) \in$$

Shultz & Co

~~pal = 0; fol = -1;~~

$$\text{del}(j = \text{del}; j \leftarrow n; j++) \in$$
$$f_{\text{col}}(k=i+1; k \leq n; k++)$$

if $f[u] = -p[u]$ break;

if $(u, \text{val}) \in$

gal -u;

pas-j;

3
[p[0] = p[0];

3
printf("Page faults = %d\n", faults);
printf("Page hits = %d\n", in-faults);
return 0;

3
Output:

Page Faults=5

Page Hits=1

Results:

Thus the optimal page replacement
~~algorithm~~ was successfully simulated.

Experiment - 34

14.08.2026

Aim:

To simulate Sequential File Allocation
using C.



Code:

It includes <stdio.h>
in main() {

int n, i, key;

printf("Enter number of records: ");

scanf("%d", &n);

int record[n];

printf("Enter records: \n");

for (i = 0; i < n; i++)

scanf("%d", &record[i]);

printf("Enter record to access: ");

scanf("%d", &key);

printf("Record found: ");

for (i = 0; i < n; i++) {

printf("%d", record[i]);

if (record[i] == key)

break;

;

if (i == n)

printf("\n Record not found");

Practical 3;

3

Input:

Enter number of records: 5

Enter records:

10 20 30 40 50

Enter record to access: 40

~~Record~~ record location: 10 20 30 40

Result:

~~As~~ Sequential File allocation was successfully implemented.

17.01.2026

Experiment-35

Aim:

To simulate the Indexed File Allocation strategy using a C program.

Code:

```
#include <stdio.h>
```

```
main() {
```

```
int n, indexBlock, block[50];
```

```
printf("Indexed File Allocation Strategy\n");
```

```
printf("Enter the index block number:");
```

```

    scanf ("%d", &indian_block); // number
    printf ("Enter the number of file blocks: ");
    scanf ("%d", &n);
    printf ("Enter the block numbers: ");
    for (int i = 0; i < n; i++) {
        printf ("Block %d: ", i+1);
        scanf ("%d", &block[i]);
    }

```

```

}
printf ("File Allocation Table\n");
printf ("-----\n");
printf ("Indian block : %d\n", indian_block);
printf ("File Block & Data Block\n");
}
return 0;

```

Output:

Enter Indian Block: 10

Enter Number of Blocks: 5

Enter Block numbers:

21
45
18
30
56

[Signature]

Index Block = 0	
File Block	Disk Block
0	21
1	45
2	18
3	30
4	56

Result:

The ~~File~~ File Allocation Strategy was successfully simulated using C.

17.08.2026

Experiment - 36

Aim:

TO Simulate the indexed file Allocation Strategy using a C program.

Code:

```
#include <stdio.h>
```

```
int main ( ) {
```

```
int n, block[20];
```

```
printf("Enter number of blocks: ");
```

```
scanf ("%d", &n);
```

```
printf("Enter block number: ");
```

```
for (int i=0; i<n; i++)
```

```
scanf ("%d", &block[i]);
```

printf ("Linker Allocation: %d",
 fd (ist 10; ist - 1, it)
 printf ("id, -> id n", block [i], head
 [i + 1]);
 printf ("id -> NULL n", block [n - 1]);
 return 0;

3

Output:

Enter Number of blocks : 5

Enter Block Number:

10

25

18

40

50

Linker Allocation:

10 -> 25

25 -> 18

18 -> 40

40 -> 50

50 -> NULL

Alt:

The ~~linked~~ File Allocation Method, is
 wellfully simulated using C program

17.05.2020

Explain the FCFS disk scheduling algorithm using a C program.

Ans: To simulate the FCFS disk scheduling algorithm using a C program.

Code:

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
int main() {
```

```
int n, head, req[20], seek = 0;
```

```
printf("Enter number of requests: ");
```

```
for (scanf("%d", &n);
```

```
printf("Enter request queue: ");
```

```
for (int i = 0; i < n; i++)
```

```
scanf("%d", &req[i]);
```

```
printf("Enter initial head position: ");
```

```
scanf("%d", &head);
```

```
for (int i = 0; i < n; i++) {
```

```
seek += abs(req[i] - head);
```

```
head = req[i];
```

```
}
```

```
printf("Total seek time = %d\n", seek);
```

Output:
Enter number of requests: 5
Enter request queue: 98 183 37 12
Enter initial head position: 53
Total seek time = 964

Result:
The SCAN Disk Scheduling algorithm was successfully simulated using C.

19-08-2026 Experiment-38

Sim:
To simulate the SCAN Disk Scheduling algorithm using a C program.

Code:

```
#include <stdio.h>
#include <stdlib.h>

int main () {
    int n, head, disk, seek = 0, req[20];

    printf("Enter number of requests: ");
    scanf("%d", &n);
```

```

printf("Enter sequence: ");
for (int i = 0; i < n; i++)
    scanf("%d", &arr[i]);

printf("Enter disk size: ");
scanf("%d", &disk);

seek = abs(arr[0] - 1 - head);
printf("Total seek time = %d\n", seek);
return 0;
}

```

Output:

Enter number of sequence: 5

Enter sequence: 98 183 37 122 14

Enter initial head position: 53

Enter disk size: 200

Total seek time = 146.

Result:

The SCAN disk scheduling algorithm
was successfully simulated using C.

19-08-2016

C++ program - 39

3

Aim:

To simulate the C-SCAN Disk Scheduling algorithm using C program.

Code:

```
#include <iostream.h>
#include <stdlib.h>
```

int main () {

int n, head, disk, seek;

printf ("Enter number of tracks: ");

scanf ("%d", &n);

printf ("Enter initial head position: ");

scanf ("%d", &head);

printf ("Enter disk size: ");

scanf ("%d", &disk);

seek = (disk - 1 - head) + (disk - 1);

printf ("Total seek time = %d\n", seek);

return 0;

Input:-

total number of requests: 5

Initial head position: 53

total disk size: 200

total seek time = 315.

Result:-

~~The~~ C-SCAN disk scheduling algorithm
was successfully simulated using C.

19-08-2026

Experiment - 40

Aim:-

To illustrate the various file access
permissions and different types of users.
Linux.

Commands:-

ls -l

chmod 755 file.txt

chmod u+x file.txt

chmod g+x file.txt

chmod o+l file.txt

chmod uel file.txt

chgrp group file.txt

Output:
- LUKL - XL - X 1 cell 120 Aug 1960
file. int
5/8 ✓

Result:
The ~~following~~ file access permissions and
different types of cells in Linu were observed
successfully.